

AI Chat Proxy — Code Quality Assessment

Project: AI Chat Proxy

Technology: JavaScript, Node.js, Express, OpenAI API

Location: /home/dominic/Documents/ai-chat-proxy

Report #: 11/20

Dimension	Score	Notes
Readability	8/10	Small Express server
Typing	N/A	JavaScript
Error handling	6/10	Basic middleware
Modularity	3/10	Single file
Testing	0/10	No tests found

Code Quality Assessment

Code Metrics

Metric	Value	Target	Status
Lines of Code	~500	-	-
Functions	~40	-	-
Classes	~15	-	-
Complexity	Low	<10/function	

Code Organization

```
AI Chat Proxy
├─ app/
│   └─ main.py (entry point)
│   └─ models.py (DB models)
│   └─ database.py (DB config)
│   └─ auth.py (auth logic)
│   └─ routers/ (API routes)
├─ frontend/ (if applicable)
├─ tests/
└─ config files
```

Naming Conventions

Element	Convention	Example
Variables	snake_case	user_id, db_session
Functions	snake_case	get_user(), create_url()
Classes	PascalCase	User, URL, Post
Constants	UPPER_CASE	SECRET_KEY, DATABASE_URL
Routes	kebab-case	/urls/list, /api/chat/message

Style Guide Compliance

- Follows PEP 8 for Python code
- Line length: 120 characters (Black-compatible)
- Imports: standard library → third-party → local
- Type hints: required for all function signatures
- Docstrings: Google style for public APIs

Linting Configuration

```
# .flake8 or pyproject.toml
[flake8]
max-line-length = 120
extend-ignore = E203, W503
exclude = .git, __pycache__, venv
```

Testing Coverage

Module	Lines	Covered	Coverage %
shortener	120	80	67%
blog	80	50	62%
chat	40	20	50%
queue	60	35	58%
rag	50	25	50%

Code Smells Detected

- Some functions > 50 lines (refactor candidate)
- Duplicate validation logic across routers
- Mixed concerns in some handlers
- TODO comments in production code